

Diplomatura en programación web full stack con React JS

Módulo 4:

Introducción React, Next y Node

Unidad 1:

React/Next (parte 1)



Presentación

En esta unidad comenzamos a conocer ReactJs / NextJs, unas librerías enfocadas en el desarrollo de interfaces.



Objetivos

Que los participantes logren...

- Saber qué es y para qué sirve ReactJs.
- Saber qué es y para qué sirve NextJs.
- Entender y utilizar el lenguaje JSX.
- Crear y utilizar nuestros propios componentes.



Bloques temáticos

1. Introducción.
2. create-next-app.
3. JSX.
4. Componentes: Props, componentes servidor, componentes cliente.

1. Introducción

ReactJS es una **librería JavaScript de código abierto** enfocada a la visualización que nos permite el desarrollo de interfaces de usuario de forma sencilla mediante componentes interactivos y reutilizables.

React está basado en un paradigma llamado programación orientada a componentes en el que cada componente es una pieza con la que el usuario puede interactuar. Estas piezas se crean usando una sintaxis llamada **JSX** **permitiendo escribir HTML (y opcionalmente CSS) dentro de objetos JavaScript.**

Estos componentes son **reutilizables** y se combinan para crear componentes mayores hasta configurar una web completa.

Esta es la forma de tener HTML con toda la funcionalidad de JavaScript y el estilo gráfico de CSS centralizado y listo para ser abstraído y usado en cualquier otro proyecto.

2. create-next-app

Next.js nos provee con una herramienta para la inicialización de nuevos proyectos llamada **create-next-app**. Esta herramienta se encargará, después de hacernos algunas preguntas sobre nuestro proyecto, de dejar creados los archivos y carpetas necesarias para empezar a trabajar.

El único requerimiento para ejecutar create-next-app es tener instalado **Node.js 18.8 o superior**.

Para inicializar nuestro proyecto escribiremos el siguiente comando en la consola:

```
npx create-next-app@latest
```

Vamos a seleccionar las siguientes opciones en las preguntas que nos irá haciendo la herramienta para obtener la configuración básica:

```
✓ Would you like to use TypeScript? ... No / Yes
✓ Would you like to use ESLint? ... No / Yes
✓ Would you like to use Tailwind CSS? ... No / Yes
✓ Would you like to use `src/` directory? ... No / Yes
✓ Would you like to use App Router? (recommended) ... No / Yes
✓ Would you like to customize the default import alias (@/*)? ... No / Yes
```

Una vez terminado el proceso podemos ingresar a la carpeta creada y empezar a trabajar en nuestro proyecto.

3. JSX

JSX es una extensión de sintaxis de Javascript, que como podremos observar se parece a HTML.

Uno de sus beneficios es **mejorar la legibilidad del código**, mejorar la experiencia del desarrollador, y reducir la cantidad de errores de sintaxis, ya que no es necesario repetir tantas veces el mismo código. Otro beneficio es el de poder integrar a otros miembros de tu equipo que no sean programadores en el desarrollo. **Es “fácil” leer el código** si ellos están acostumbrados a leer HTML.

React no requiere usar JSX, pero la mayoría de la gente lo encuentra útil como ayuda visual cuando trabajan con interfaz de usuario dentro del código Javascript. Esto también permite que React muestre mensajes de error o advertencia más útiles.

Insertando expresiones en JSX

```
1 const name = 'Josh Perez';
2 const element = <h1>Hello, {name}</h1>;
3
4 ReactDOM.render(
5   element,
6   document.getElementById('root')
7 );
```

En el ejemplo declaramos una variable llamada name y luego la usamos dentro del JSX envolviéndola **dentro de llaves**.


```
1 function formatName(user) {
2   return user.firstName + ' ' + user.lastName;
3 }
4
5 const user = {
6   firstName: 'Harper',
7   lastName: 'Perez'
8 };
9
10 const element = (
11   <h1>
12     Hello, {formatName(user)}!
13   </h1>
14 );
15
16 ReactDOM.render(
17   element,
18   document.getElementById('root')
19 );
```

En el ejemplo insertamos el resultado de llamar la función de JavaScript, `formatName(user)`, dentro de un elemento `<h1>`.

Especificando atributos con JSX

Podemos utilizar comillas para especificar strings literales como atributos:

```
1 const element = <div tabIndex="0"></div>;
```

También puedes usar llaves para insertar una expresión JavaScript en un atributo:

```
1 const element = <img src={user.avatarUrl}></img>;
```

Podemos usar comillas rodeando llaves cuando insertes una expresión JavaScript en un atributo. Debemos utilizar comillas (para los valores de los strings) o llaves (para las expresiones), pero no ambas en el mismo atributo.

Advertencia:

Dado que JSX es más cercano a JavaScript que a HTML, React DOM usa la convención de nomenclatura camelCase en vez de nombres de atributos HTML.

Por ejemplo, **class** se vuelve **className** en JSX, y **tabindex** se vuelve **tabIndex**.

4. Components: Props, Componentes servidor, Componentes cliente

Props

Las propiedades (o **props** de aquí en adelante) son valores que se pasan a los componentes para modificar su comportamiento o valor de retorno. Se especifican el elemento de la misma forma que un atributo HTML.

Como vimos previamente los valores de las **props** pueden ser valores literales como strings pasados entre comillas, expresiones de JavaScript, referencias a funciones u otros componentes.

Ya sea que declares un componente como una función o como una clase, este nunca debe modificar sus props. Considera esta función sum:

```
1 function suma(a, b) {  
2   return a + b;  
3 }
```

Tales funciones son llamadas “puras” porque no tratan de cambiar sus entradas, y siempre devuelven el mismo resultado para las mismas entradas.

En contraste, esta función es impura porque cambia su propia entrada:

```
1 function retiro(cuenta, monto) {  
2   cuenta.total -= monto;  
3 }
```

React es bastante flexible pero tiene una sola regla estricta: **Todos los componentes de React deben actuar como funciones puras con respecto a sus props.**

Componentes de servidor

Por defecto todos los componentes como layouts y paginas son interpretados como **componentes de servidor**. Esto quiere decir que Next.js va a consultar las fuentes de datos necesarias y procesar el resultado final del renderizado de los componentes para enviarlos al navegador del usuario en su forma final. Esta metodología, mas allá de reducir el procesamiento que se realiza en el navegador, permite implementar distintas estrategias de cache y entrega de los datos, lo que resulta en sitios mucho mas responsivos y ágiles para el usuario.

Los componentes de servidor son ideales para realizar consultas de datos, ya sea a bases de datos o APIs. Al ejecutarse en el servidor podemos usar libremente credenciales de acceso o tokens sin que estos se vean expuestos al usuario. Todo esto reduce, como dijimos anteriormente, la cantidad de código Javascript enviada al navegador para su posterior procesamiento, incrementando la velocidad de respuesta y mejorando la experiencia del usuario.

No es necesaria ninguna configuración ni código adicional para que nuestros componentes funcionen como componentes de servidor.

Componentes de cliente

Al contrario de los componentes de servidor, los componentes de cliente se ejecutan exclusivamente en el navegador del usuario. Esto nos permite tener acceso a todas las APIs del navegador como eventos (onClick, onChange, etc), localStorage, geolocalización, etc.

También, al ejecutarse en el navegador, tendremos acceso a los hooks de React como el useState o useEffect entre otros.

Para convertir un componente en componente de cliente solo bastará con agregar la directiva **“use client”** como **primera línea del archivo**.

Estado

El estado es el **“corazón”** de los componentes de React. Es la característica que te permitirá desarrollar aplicaciones mucho más interesantes. Un estado en React es, entonces, un almacén de datos mutable de componentes y que además son autónomos.

Diferencia entre estado y props

Los estados actúan en el contexto del componente y por otro, las propiedades crean una instancia del componente cuando le pasas un nuevo valor desde un componente padre.

Los valores de las propiedades los pasas de padres a hijos y los valores de los estados los defines en el componente, no se inician en el componente padre.

Estado en clases

```
1 class App extends React.Component {
2   constructor(props) {
3     super(props);
4     this.state = {date: new Date()};
5   }
6
7   render() {
8     return (
9       <div>
10        Hoy es{this.state.date.toLocaleTimeString()}
11      </div>
12    );
13  }
14 }
```

En los componentes de clase el estado se define como una propiedad de la clase llamada state representada por un objeto literal de Javascript, la cual podemos modificar mediante el uso del método setState que todos los componentes que extienden a React.Component tienen.

Estado en componentes funcionales

```
1 import React, { useState } from "react";
2
3 const App = (props) => {
4
5   const [hoy, setHoy] = useState(new Date());
6
7   return (
8     <div>
9       Hoy es {hoy.toLocaleDateString()}
10    </div>
11  );
12 }
13
14 export default App;
```

En el caso de los componentes funcionales podemos hacer uso del hook `useState` para definir una variable local a nuestro componente. Dicha variable podría ser modificada con la función asociada a la misma (en este caso `setHoy`).

Hooks

Hooks son una característica que apareció en React 16.8. Estos te permiten usar el estado y otras características de React sin escribir una clase.

Los hooks son básicamente **funciones especiales** que permiten “conectarnos” a distintas características de React y permiten su reutilización entre componentes. Por ejemplo usando componentes de clase, la lógica de manejo de estado sería muy difícil (sino imposible) de reutilizar en otros componentes. Creando nuestros propios hooks podríamos compartir esta funcionalidad con todos los componentes que la necesitaran.

Igualmente vamos a centrarnos en 2 de los hooks que incluye React que utilizaremos más a menudo.

Ejemplo useState

En una clase, inicializamos el estado **count** a 0 estableciendo `this.state` a **{ count: 0 }** en el constructor:

```
1 class Example extends React.Component {  
2   constructor(props) {  
3     super(props);  
4     this.state = {  
5       count: 0  
6     };  
7   }
```

En un componente funcional no existe `this` por lo que no podemos asignar o leer **this.state**. En su lugar, usamos el Hook **useState** directamente dentro de nuestro componente.


```
1 import React, { useState } from 'react';  
2  
3 function Example() {  
4   // Declaración de una variable de estado que llamaremos  
   "count"  
5  
6   const [count, setCount] = useState(0);
```

¿Qué hace la llamada a useState? Declara una “variable de estado”. Nuestra variable se llama **count**, pero podemos llamarla como queramos, por ejemplo banana. Esta es una forma de “preservar” algunos valores entre las llamadas de la función - **useState** es una nueva forma de usar exactamente las mismas funciones que **this.state** nos da en una clase. Normalmente, las variables “desaparecen” cuando se sale de la función, pero las variables de estado son conservadas por React

¿Qué pasamos a useState como argumento? El único argumento para el Hook **useState()** es el estado inicial. Al contrario que en las clases, el estado no tiene porque ser un objeto. Podemos usar números o strings si es todo lo que necesitamos. En nuestro ejemplo, solamente queremos un número para contar el número de clicks del usuario, por eso pasamos 0 como estado inicial a nuestra variable. (Si queremos guardar dos valores distintos en el estado, llamaríamos a **useState()** dos veces)

¿Qué devuelve useState? Devuelve una pareja de valores: el estado actual y una función que lo actualiza. Por eso escribimos **const [count, setCount] = useState()**. Esto es similar a **this.state.count** y **this.setState** en una clase, excepto que se obtienen juntos. Si no conoces la sintaxis que hemos usado volveremos a ella al final de esta página.

Ahora que sabemos que hace el Hook **useState**, nuestro ejemplo debería tener más sentido:

```
1 import React, { useState } from 'react';  
2  
3 function Example() {  
4   // Declaración de una variable de estado que llamaremos  
   "count"  
5  
6   const [count, setCount] = useState(0);
```

Declaramos una variable de estado llamada **count** y le asignamos a 0. React recordará su valor actual entre re-renderizados, y devolverá el valor más reciente a nuestra función. Si se quiere actualizar el valor de count actual, podemos llamar a setCount

useEffect

El Hook de efecto te permite llevar a cabo efectos secundarios en componentes funcionales:

```
1 import React, { useState, useEffect } from 'react';
2
3 function Example() {
4   const [count, setCount] = useState(0);
5
6   // De forma similar a componentDidMount y componentDidUpdate
7   useEffect(() => {
8     // Actualiza el título del documento usando la API del
    navegador
9     document.title = `You clicked ${count} times`;
10  });
11
12  return (
13    <div>
14      <p>You clicked {count} times</p>
15      <button onClick={() => setCount(count + 1)}>
16        Click me
17      </button>
18    </div>
19  );
20 }
```

Este fragmento está basado en el ejemplo del contador anterior, pero le hemos añadido una funcionalidad nueva: actualizamos el título del documento con un mensaje personalizado que incluye el número de clicks.

Peticiones de datos, establecimiento de suscripciones y actualizaciones manuales del DOM en componentes de React serían ejemplos de efectos secundarios. Tanto si estás acostumbrado a llamar a estas operaciones “efectos secundarios” (o simplemente “efectos”) como si no, probablemente los has llevado a cabo en tus componentes con anterioridad.

Por defecto se ejecuta después del primer renderizado y después de cada actualización.

Si queremos controlar cuando se ejecuta el hook `useEffect` podemos pasarle como 2do parámetro un array con los valores que deseamos monitorear.

```
1 const Ejemplo = (props) => {
2
3   const [count, setCount] = useState(0)
4
5   useEffect(() => {
6     // código del hook
7   }, [count])
8
9 });
10 }
```

Si pasamos un array vacío, solo se ejecutará el código del hook cuando el componente se incluya en el documento por primera vez. Este modo es muy útil para cargar datos desde el servidor cuando se incluye un componente.

```
1 const Ejemplo = (props) => {
2
3   useEffect(() => {
4     console.log('Se agregó Ejemplo');
5   }, [])
6
7 }
```

Si necesitamos ejecutar una función al eliminar un componente podemos incluir ese código dentro de una función que sea devuelta por el hook

```
1 const Ejemplo = (props) => {  
2  
3   useEffect(() => {  
4     console.log('Se agregó Ejemplo');  
5  
6     return () => console.log('Se quitó Ejemplo')  
7   }, [])  
8  
9 }
```

Eventos

Los eventos en React se manejan de forma similar a los eventos de HTML con algunas diferencias:

- En React los eventos se nombran con camelCase en vez de todo minúscula.
- JSX permite pasar una función como manejador del evento en vez de un string como HTML.
- En React no podemos devolver **false** para prevenir el comportamiento por defecto del evento. Debemos llamar explícitamente al método **preventDefault()**.

Ejemplo de manejo de click



```
1 const MiBoton = (props) => {  
2   const [prendido, setPrendido] = useState(false);  
3  
4   return (  
5     <button onClick={() => setPrendido(!prendido)}>  
6       {prendido ? 'Apagar' : 'Prender'}  
7     </button>  
8   );  
9 }
```

Conclusión

En resumen, aprendimos los conceptos básicos de React y Next, sus tipos de componentes, cómo y cuándo utilizarlos y como pasarles parámetros que modifiquen su comportamiento o respuesta. También aprendimos sobre el estado de los componentes y el manejo de eventos en los mismos.

En la próxima unidad veremos...

En resumen, aprendimos los conceptos básicos de React y Next, sus tipos de componentes, cómo y cuándo utilizarlos y como pasarles parámetros que modifiquen su comportamiento o respuesta. También aprendimos sobre el estado de los componentes y el manejo de eventos en los mismos.



Bibliografía utilizada y sugerida

Libros y otros manuscritos:

Alex Banks & Eve Porcello. Learning React: desarrollo web funcional con React y Redux. 2017.

Artículos de revista en formato electrónico:

Hoyos, S. (2017, octubre 23). Enrutando en React. Medium.

<https://medium.com/@simonhoyos/enrutando-en-react-cd9e4ad6e3d3>

MDN Web Docs. (s.f.). MDN Web Docs. Mozilla. <https://developer.mozilla.org/>

React. (s.f.). React: Biblioteca de JavaScript para construir interfaces de usuario. Meta.

<https://es.reactjs.org/>

Vercel. (s.f.). Next.js: The React Framework for Production. <https://nextjs.org/>